



Use subqueries to aggregate data

Hey, there. We've got some experience nesting subqueries into our SQL statements to perform more complicated queries. Now it's time to talk about how to aggregate data with subqueries. Coming up, we'll learn about some new subquery statements and use them to aggregate data. The query we're going to build in this video is pretty advanced. It's going to be a little complicated, but I know you've got what it takes. Let's get started. We've used functions like WHERE to filter our data before, but the WHERE function can't be used with aggregate functions. For example, you can use WHERE on a statement and follow it with GROUP BY. But when you want to use GROUP BY first and then use WHERE on that output, you'll need a different function. This is where HAVING comes in. HAVING basically allows you to add a filter to your query instead of the underlying table when you're working with aggregate functions. That way it only returns records that meet your specific conditions. Similarly, CASE returns records with your conditions by allowing you to include if/then statements in your query.

Let's try to aggregate our data with subqueries and test out these new functions. Let's say we're working with a company that makes socks that we talked about earlier. We've been asked to calculate what percentage of the orders are fulfilled by each warehouse. Basically, we're interested in knowing which warehouses are delivering the most orders.

We've seen these tables before, but as a quick refresher, here's the Orders table. You can see the columns here: `order_id`, `customer_id`, `warehouse_id`, `order_date`, and `ship_date`. If we pull up the warehouse table, we can check out its columns.

We have the `warehouse_id`, `warehouse_alias`, the `maximum_capacity`, the total number of employees, and the state where the warehouse is located. Before we start building the rest of our query, we'll want to alias our table names. As a reminder, aliasing is when you temporarily name a table or column in your query to make it easier to read and write. This example query is a little bit more complicated than the ones we've seen before. Aliasing will help save us some time. We'll start by aliasing the Warehouse table in our FROM statement. The FROM statement in this query is near the end, but we'll build this first so that we can use the alias everywhere else.

We'll simplify it to just Warehouse for the rest of this query. We know that we're going to JOIN these tables together. Let's add that while we're working on this part of the query anyway. We're using a LEFT JOIN here because we want all the information from our Warehouse data, even if it doesn't show up in the Orders table. Then we'll alias the Orders table as part of this statement. Now both of our tables have temporary names we can use. We've already finished a JOIN statement.

But before we can build the beginning of this query, let's go ahead and add our GROUP BY statement after this JOIN. We'll group these by the `warehouse_id` and name. Now we'll go back to the beginning of the query. We'll select the `warehouse_id`. Then we'll use CONCAT to combine the strings with the warehouse's state and alias AS the warehouse name. Then we'll use COUNT to get the number of orders per warehouse.

Next, we'll build in a subquery to pull the total number of orders placed across all warehouses.

We'll input SELECT again and then write the subquery in parentheses. We'll put an asterisk after COUNT to indicate that we want to include everything from the Orders table. Finally, we'll close out the subquery and use AS to name this column total_orders.

Now that our subquery logic is complete, we can use a CASE statement to create categories for our warehouses based on how many orders they will fulfill. We'll represent these as percentages. You should notice COUNT in the statement a few times. We'll start by saying WHEN the number of orders FROM our Orders table is less than or equal to 0.2, THEN the table will say "Fulfilled 0-20% of Orders."

Then we'll use WHEN again to indicate that when the number of orders is greater than 0.2 and less than or equal to 0.6, it'll say, "fulfilled 21-60% of Orders." After that, we can use ELSE to have everything that doesn't meet the criteria of our CASE statement say, "Fulfilled more than 60% of Orders." Then we'll use END AS to name this column fulfillment_summary. That brings us back to the portion of the query we've already written. But we're going to add a HAVING statement at the very end of this query. Our Warehouse table has warehouses that are currently being built, and we want to filter those out since they aren't fulfilling orders yet. We can use HAVING to only include warehouses that have at least one order.

Now, before we execute this query, let's take a moment to look at the whole thing. We have an outer SELECT, a COUNT subquery, a CASE statement, a JOIN and HAVING, all wrapped into one query. We've built a really complex query. So let's run it to see the

new table. There. Now we can easily identify what percent of our company's total orders are being fulfilled by each warehouse. These warehouses met our criteria.

And we can see here in the fulfillment_summary column the percentage categories we outlined in our CASE statement. Obviously, since we included a HAVING statement to specify only warehouses with at least one order, there aren't any warehouses currently under construction in this table. That really complicated query we wrote created this specific table of data we can use to easily compare how these warehouses are performing. There you go. That's a quick taste of what it's like to work with subqueries and data aggregation. Clauses like HAVING and CASE paired with subqueries will help you build more and more complex queries, which lets you do more and more complex things in SQL.